Introdução ao Maxima

Sistema de Álgebra Computacional (CAS)

Eduardo Lenz

10 de março de 2026

O que é um Sistema de Álgebra Computacional (CAS)?

- Programas tradicionais (como C, Fortran ou MATLAB) lidam essencialmente com **computação numérica** (aproximações de ponto flutuante).
- Um **CAS** (Computer Algebra System) é desenhado para manipular expressões matemáticas de forma **simbólica** e **exata**.
- **Vantagens:**
  - Resolve integrais e derivadas analiticamente (retorna a equação, não apenas um número).
  - Simplifica expressões algébricas complexas.
  - Trabalha com precisão infinita (ex: frações exatas como 1/3 em vez de 0.3333...).

Um Breve Histórico do Maxima

- O Maxima é um descendente direto do **Macsyma**, o lendário CAS desenvolvido no MIT (Project MAC) no final dos anos 1960.
- Foi financiado pelo Departamento de Energia dos EUA (DOE), originando a versão DOE-Macsyma.
- Mantido pelo Professor **William Schelter** da Universidade do Texas de 1982 até sua morte em 2001.
- Em 1998, Schelter conseguiu permissão do DOE para lançar o código-fonte sob a licença **GPL (Open Source)**. Nasceu assim o **Maxima**.
- Hoje, é ativamente mantido por uma comunidade global e frequentemente utilizado através da interface gráfica **wxMaxima**.

Sintaxe Básica: Finalizadores e Atribuições

Finalizadores de Linha

Toda instrução no Maxima deve terminar com ; ou $.

- ; executa o comando e **mostra** o resultado na tela.
- $ executa o comando, mas **oculta** o resultado (útil para passos intermediários longos).

Atribuição vs. Igualdade

- **Atribuição (:)**: Armazena um valor em uma variável.
  Ex: `a : 5;` (A variável 'a' agora vale 5).
- **Igualdade (=)**: Representa uma equação matemática, não altera valores na memória.
  Ex: `eq1 : x^2 + y = 10;` (Guarda a equação na variável 'eq1').

Constantes e Variáveis Embutidas

O Maxima possui constantes matemáticas pré-definidas. Elas sempre começam com o símbolo de porcentagem (%) para evitar conflitos com variáveis do usuário.

- `%pi`: O número π (3.14159...).
- `%e`: A base dos logaritmos naturais (Número de Euler).
- `%i`: A unidade imaginária ($\sqrt{-1}$).

Exemplo de uso:

```
angulo : %pi / 4;
z : 3 + 4*%i;
```

Substituição e Avaliação de Expressões

Muitas vezes queremos avaliar uma equação simbólica para valores específicos sem reescrever a equação inteira.

Comando subst

Substitui uma parte da expressão por outra. Sintaxe: `subst(novo, antigo, expressao);`

```
expr : x^2 + 2*x + 1;
subst(3, x, expr);      /* Retorna: 3^2 + 2*3 + 1 -> 16 */
```

Comando ev

Avalia (Evaluate) uma expressão aplicando um ambiente de variáveis.

```
expr : m * a;
ev(expr, m=10, a=9.8); /* Retorna: 98.0 */
```

Vetores e Matrizes

- **Listas/Vetores:** Definidos entre colchetes.

```
v : [1, 2, 3];
```

- **Matrizes:** Usando o comando `matrix`, onde cada argumento é uma linha (lista).

```
M : matrix( [1, 2], [3, 4] );
```

Multiplicação de Matrizes

Use o **ponto (.)** para a álgebra linear real (linha x coluna).
O asterisco (*) multiplica elemento a elemento!

```
resultado : M . v;
transposta : transpose(M);
inversa : invert(M);
```

Cálculo Simbólico

Derivadas (diff)

Sintaxe: `diff(expressão, variável, [ordem]);`

```
f : sin(x) * exp(x);
diff(f, x);       /* Primeira derivada em relação a x */
diff(f, x, 2);    /* Segunda derivada em relação a x */
```

Integrais (integrate)

Indefinida: `integrate(expressão, variável);`

Definida: `integrate(expressão, variável, limite_inf, limite_sup);`

```
integrate(x^2, x);          /* Retorna x^3 / 3 */
integrate(x^2, x, 0, 1);    /* Retorna 1/3 */
```

Gráficos: O Pacote `draw`

O Maxima possui um pacote avançado chamado `draw` que interage com o Gnuplot. É necessário carregá-lo antes de usar com `load(draw);`.

Gráficos Explícitos (`explicit`)

Usado quando temos $y = f(x)$.

```
load(draw);
draw2d(
    color = blue,
    explicit(x^2 - 4, x, -5, 5)
);
```

Gráficos: Funções Implícitas

Gráficos Implícitos (`implicit`)

Usado para equações onde x e y estão misturados, como circunferências ($x^2 + y^2 = r^2$).

```
draw2d(
    color = red,
    line_width = 2,
    implicit(x^2 + y^2 = 9, x, -4, 4, y, -4, 4)
);
```

Dica: O `draw3d` funciona de maneira semelhante para superfícies tridimensionais.

Programação: Subrotinas e Variáveis Locais

- Definimos funções (subrotinas) no Maxima usando o operador `:=`.
- Para rotinas complexas com múltiplas linhas, usamos a estrutura `block`.
- A primeira lista dentro do `block` declara as **variáveis locais** (que deixam de existir quando a função termina, economizando memória e evitando conflitos).

```
distancia(x1, y1, x2, y2) := block(
    [ dx, dy ], /* Variáveis locais */

    dx : x2 - x1,
    dy : y2 - y1,

    return( sqrt(dx^2 + dy^2) )
)$
```

Programação: Condicionais e Laços de Repetição

Estrutura Condicional (if)

```
if (x > 0) then (
    print("Positivo")
) else (
    print("Negativo ou zero")
);
```

Laços de Repetição (for)

Sintaxe básica para iterar contadores:

```
soma : 0;
for i from 1 thru 10 do (
    soma : soma + i
);
```